

Descripción y justificación del problema

1. Contexto del problema

Las aplicaciones web distribuidas modernas estan compuestas por multiples componentes independientes: servidores de backend, bases de datos, API Gateways, servicios de autentificacion y capas de red. Cuando ocurre una falla, los sintomas que percibe el usuario (mensajes de error, lentitud, imposibilidad de iniciar sesion) rara vez apuntan directamente a una unica causa.

Por ejemplo:

- Un **error 500** puede originarse por una base de datos caída, un backend saturado o una falla interna del servidor.
- Una **latencia alta** puede deberse a una conexión de red inestable, a un gateway colapsado o a un backend sobrecargado.
- Un **login fallido** puede ser consecuencia directa de un error en el servicio de autentificación, pero también podría coincidir con otras fallas del sistema.

Esta ambigüedad hace que el diagnostico manual sea lento e impreciso, especialmente cuando varios sintomas aparecen al mismo tiempo y pueden tener causas comunes o independientes.

2. Descripción del problema

El problema central del proyecto es el siguiente:

Dado un conjunto de sintomas observados en una aplicacion web distribuida, determinar cuales son las causas mas probables de la falla.

Formalmente, se busca calcular distribuciones de probabilidad de la forma:

$$P(\text{causa} \mid \text{sintoma}_1 = \text{valor}_1, \text{sintoma}_2 = \text{valor}_2, \dots)$$

Por ejemplo:

```
P(BaseDatosCaída | Error500 = si, LatenciaAlta = si, LoginFallido = no)
P(BackendSaturado | Error500 = si, ServicioNoDisponible = si, LatenciaAlta = si)
P(GatewayCaído | ServicioNoDisponible = si, LatenciaAlta = si, Error500 = no)
```

El sistema no se conecta a una aplicación real. En cambio, modela el comportamiento probabilístico de los componentes mediante una red bayesiana construida a partir de conocimiento experto sobre como se relacionan causas y sintomas en sistemas web típicos.

3. Variables del modelo

El modelo distingue dos tipos de variables booleanas (pueden ser **si** o **no**):

Variables causa

Representan posibles fallas internas en los componentes del sistema:

Variable	Significado	P(si) a priori
BaseDatosCaida	La base de datos presenta una falla activa	0.05
BackendSaturado	El servidor backend esta sobrecargado	0.10
GatewayCaído	El API Gateway no esta respondiendo	0.04
ConexionInestable	La red presenta intermitencia o perdida de paquetes	0.12
ErrorAutenticacion	El servicio de autenticacion tiene una falla activa	0.08

Variables sintoma

Representan efectos observables que el usuario o el sistema de monitoreo pueden detectar:

Variable	Significado	Causas relacionadas
Error500	Error HTTP 500 en respuestas del servidor	BaseDatosCaida, BackendSaturado
LatenciaAlta	Tiempos de respuesta superiores al umbral normal	GatewayCaído, ConexionInestable
LoginFallido	Fallo en el proceso de inicio de sesion	ErrorAutenticacion
ServicioNoDisponible	El servicio no responde en absoluto	GatewayCaído, BackendSaturado, BaseDatosCaida
UsuarioAfectado	El usuario final percibe degradacion del servicio	Error500, LatenciaAlta

4. Por que este problema pertenece al area de computacion

Este problema pertenece al area de computacion por las siguientes razones:

4.1 Requiere razonamiento automatico bajo incertidumbre

Los sistemas de diagnostico no pueden basarse en reglas deterministas del tipo "si hay Error500 entonces la base de datos caida", porque la misma causa puede no producir siempre el mismo sintoma, y el mismo sintoma puede tener multiples causas posibles. La solucion formal a este tipo de problemas es el razonamiento probabilistico, un campo central de la inteligencia artificial.

4.2 Implementa un modelo de IA del capitulo 13 de Russell & Norvig

El proyecto aplica directamente los conceptos de redes bayesianas e inferencia por enumeracion presentados en *Artificial Intelligence: A Modern Approach* (capitulo 13). Esto lo ubica en el area de agentes inteligentes que

razonan con conocimiento incierto, un tema fundamentalmente computacional.

4.3 El algoritmo de inferencia no es trivial

Calcular $P(\text{causa} \mid \text{evidencia})$ sobre una red con 10 variables booleanas requiere sumar sobre combinaciones exponenciales de variables ocultas. El algoritmo de inferencia por enumeracion implementado en `inference.py` recorre el espacio de estados de forma recursiva siguiendo el orden topologico de la red, una tecnica que solo es practicable con una implementacion computacional.

4.4 El modelo captura dependencias causales complejas

La estructura de la red refleja relaciones causales reales entre componentes de software: un nodo como `ServicioNoDisponible` tiene tres causas simultaneas (`GatewayCaído`, `BackendSaturado`, `BaseDatosCaída`), lo que hace que el diagnostico manual sea propenso a errores. El sistema computa automaticamente como cada evidencia actualiza las probabilidades de todas las causas a la vez, aprovechando la independencia condicional para hacerlo de forma eficiente.

5. Alcance del proyecto

El proyecto cubre los siguientes aspectos:

- Modelado de una red bayesiana con 5 nodos causa y 5 nodos sintoma.
- Construcción manual de tablas de probabilidad condicional (CPT) para cada nodo.
- Implementación desde cero del algoritmo de inferencia por enumeración (sin librerías externas de IA).
- Cálculo de consultas probabilísticas a partir de evidencia observada.
- Normalización de resultados para obtener distribuciones de probabilidad válidas.
- Ejecución de 5 escenarios de prueba representativos con análisis de sensibilidad.
- Interfaz de consola interactiva para consultas manuales y escenarios automatizados.
- Generación de gráficas comparativas de resultados.

El proyecto **no** cubre:

- Conexión con sistemas reales de monitoreo o logs.
- Aprendizaje automático de parámetros CPT a partir de datos.
- Algoritmos de inferencia aproximada (como muestreo de Monte Carlo).
- Despliegue como servicio web o API REST.

Estas limitaciones son intencionales: el objetivo académico es demostrar el dominio de los conceptos del capítulo 13 mediante una implementación limpia y comprensible, no construir un sistema de producción.

6. Justificación de las probabilidades a priori

Las probabilidades a priori asignadas a los nodos causa reflejan la frecuencia esperada de fallas en un sistema web típico bajo operación normal:

- `BaseDatosCaída` = 0.05: las fallas de base de datos son poco frecuentes pero tienen alto impacto.
- `BackendSaturado` = 0.10: los picos de carga son más comunes, especialmente en sistemas sin autoscaling.
- `GatewayCaído` = 0.04: el API Gateway suele ser el componente más estable por su simplicidad.

- `ConexionInestable = 0.12`: los problemas de red son frecuentes en entornos de nube o con dependencias externas.
- `ErrorAutenticacion = 0.08`: los errores de autenticacion tienen causas diversas (tokens expirados, configuraciones incorrectas).

Estas probabilidades no provienen de datos estadísticos reales sino de criterio experto razonado. Su coherencia se verifica en el análisis de sensibilidad de `scenarios.py`, donde se comprueba que la evidencia actualiza las probabilidades en la dirección esperada.

7. Conexion con los objetivos especificos

Objetivo especifico	Como lo aborda el sistema
Modelar fallas con red bayesiana	<code>model.py</code> construye la red con 10 nodos y sus dependencias causales
Definir variables booleanas	Todas las variables son <code>True/False</code> implementadas en la clase <code>Node</code>
Construir tablas CPT	Cada nodo en <code>model.py</code> tiene su CPT con valores justificados
Implementar inferencia por enumeracion	<code>inference.py</code> implementa <code>enumeration_ask</code> y <code>enumerate_all</code> desde cero
Calcular consultas <code>P(X</code>	<code>evidencia)</code>
Normalizar resultados	La funcion <code>normalize</code> en <code>inference.py</code> garantiza que la suma sea exactamente 1.0
Probar con escenarios representativos	<code>scenarios.py</code> contiene 5 escenarios completos con analisis de sensibilidad
Analizar cambios segun evidencia	La funcion <code>compare_prior_vs_posterior</code> muestra el impacto de cada escenario